

LINQ Take Method in C# with Examples

In this article, I will discuss the **LINQ Take Method in C#** with Examples. Please read our previous article before proceeding to this article, where we briefly introduced [LINQ Partitioning Methods](#). As part of this article, we will discuss the following pointers related to the Take Method in C#.

1. [What is the Take Method in C#?](#)
2. [Multiple Examples to Understand Take Method using both Method and Query Syntax.](#)
3. [LINQ Take Method using Filter in C#.](#)
4. [What happens when Applying the Where Method after the Take Method?](#)
5. [LINQ Take Method on NULL Data Source.](#)
6. [Examples to Understand Take Method with Complex Type.](#)
7. [When to Use LINQ Take Method in C#?](#)

What is the LINQ Take Method?

The LINQ Take method in C# is used to fetch a specified number of elements from the start of a collection. It is part of the System.Linq namespace and can be used with any IEnumerable<T> or IQueryable<T> sequence. Take is particularly useful in scenarios like pagination, where you want to retrieve only a subset of elements.

The **LINQ Take Method in C#** fetches the first “n” number of elements from the data source, where “n” is an integer passed as a parameter to the LINQ Take method. Only one version is available for this LINQ Take Method in C#, and the method signature is shown below.

```
public static IEnumerable<TSource> Take<TSource>(this IEnumerable<TSource>
source, int count);
```

As you can see in the above signature, the LINQ Take method takes one integer count parameter, which will return that number of contiguous elements from the data source. If the data source is null, it will throw an exception. We can use the LINQ Take method with both Method and Query Syntax. But the most important point that you need to remember is that it will not make any changes to the positions of the elements in the original data source.

Characteristics:

- **Deferred Execution:** Like many LINQ methods, Take uses deferred execution. The elements are not retrieved until you iterate over the result.
- **Limiting Results:** It limits the number of results to the specified count. If the original sequence contains fewer elements than the requested count, it returns all available elements without throwing an error.
- **Order Preservation:** Take maintains the order of the elements as they appear in the original sequence.

Examples to Understand LINQ Take Method in C#:

Let us see an example of the LINQ Take Method in C# using Method and Query Syntax. Please have a look at the following example. In the following example, we created one integer collection which contains 10 elements. Then, we retrieve the first 4 elements from the collection using the LINQ Take method with both Method syntax and Query Syntax. But you need to remember that no such operator called take is available in LINQ to write the query syntax. So, here, we need to use the mixed syntax shown in the example below.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQTakeMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Sequence Contains 10 Elements
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            //Fetching the First four elements from the Sequence using Take Method
            //Using Method Syntax
            List<int> ResultMS = numbers.Take(4).ToList();
            //Using Query Syntax
            List<int> ResultQS = (from num in numbers
                                select num).Take(4).ToList();
            //Accessing the Results using Foreach Loop
            foreach (var num in ResultMS)
            {
                Console.WriteLine($"{num} ");
            }
            Console.ReadKey();
        }
    }
}
```

Output: 1 2 3 4

Use Cases of LINQ Take Method in C#:

- Pagination: Fetching a specific number of items for a page display in web or desktop applications.
- Sampling: Taking a sample from a larger dataset for testing or analysis.
- Limiting Output: When you only need the first few elements from a collection, such as the top N items.

LINQ Take Method with Filtering Method in C#:

Now, let us understand how we can use the LINQ Take method with the LINQ Where Extension Method and what is the better approach to use these two methods. Our requirement is to take the first four elements from the collection, but the condition is that the elements should be greater than 3. In such cases, first, we need to filter the data using the Where Extension Method, and then we need to take the first four elements using the LINQ Take Method. For a better understanding, please have a look at the following example.

```
using System;
```

```

using System.Collections.Generic;
using System.Linq;
namespace LINQTakeMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Sequence Contains 10 Elements
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            //Fetching the First four elements from the Sequence where Number is greater than 3
            //Using Method Syntax
            List<int> ResultMS = numbers.Where(num => num > 3).Take(4).ToList();

            //Using Query Syntax
            List<int> ResultQS = (from num in numbers
                                where num > 3
                                select num).Take(4).ToList();

            //Accessing the Results using Foreach Loop
            foreach (var num in ResultMS)
            {
                Console.WriteLine($"{num} ");
            }
            Console.ReadKey();
        }
    }
}

```

Output: 4 5 6 7

Where Method After Take Method in C#:

If you apply the Filtering Operator, i.e., the Where condition after the Take method, you will not get the result as expected as shown in the example below.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQTakeMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Sequence Contains 10 Elements
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            //Fetching the First four elements from the Sequence where Number is greater than 2
            //Using Method Syntax
            List<int> ResultMS = numbers.Take(4).Where(num => num > 2).ToList();

            //Using Query Syntax

```

```

        List<int> ResultQS = (from num in numbers
                             select num)
                             .Take(4)
                             .Where(num => num > 2)
                             .ToList();
        //Accessing the Results using Foreach Loop
        foreach (var num in ResultMS)
        {
            Console.Write($"{num} ");
        }
        Console.ReadKey();
    }
}

```

Output: 3 4

As you can see in the above output, we are getting only 2 elements. This is because of the wrong use of the Where Filtering Method. In this case, first, the “**Take**” method is executed, which will return four elements, and then, on this result set, the “**Where**” method is applied, further filtering the data based on the specified condition. So, using the Where Method first and then the Take Method is always a good programming practice.

What happens when the Data Source is null?

When we apply the Take Method on a data source that is null, we will get an exception, i.e., **ArgumentNullException**, as shown in the example below.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQTakeMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Source is Null
            List<int> numbers = null;
            //Fetching the First four elements from the Sequence
            //Using Method Syntax
            List<int> ResultMS = numbers.Take(4).ToList();
            //Using Query Syntax
            List<int> ResultQS = (from num in numbers
                                 select num)
                                 .Take(4)
                                 .ToList();
            //Accessing the Results using Foreach Loop
            foreach (var num in ResultMS)
            {

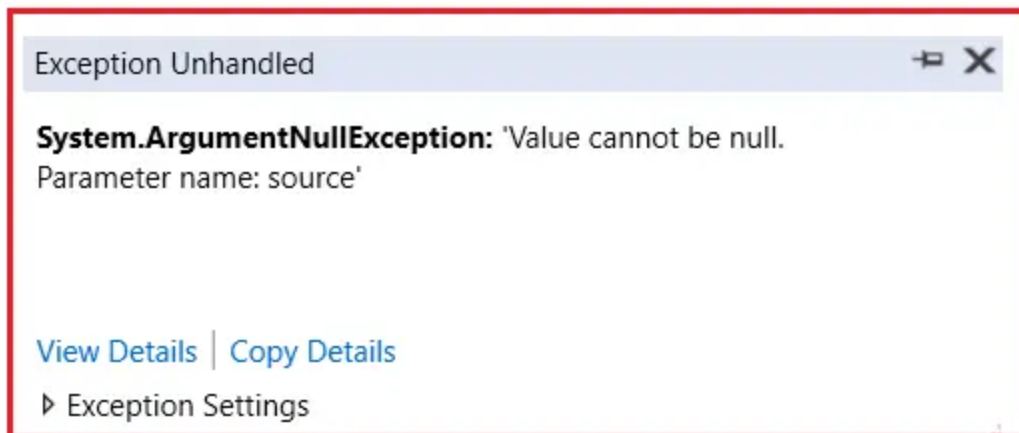
```

```

        Console.WriteLine($"{num} ");
    }
    Console.ReadKey();
}
}
}

```

When you run the application, you will get the following exception.



Example to Understand LINQ Take Extension Method with Complex Types in C#:

Let us see how to use the LINQ Take Extension Method with Complex Data Types using C#. We are going to use the following Employee class. So, create a class file named **Employee.cs** and copy and paste the following code. As you can see, here we created the Employee class with four properties, i.e., ID, Name, Gender, and Salary. Here, we have also created one method which will return the list of all employees which will be going to our data source.

```

using System.Collections.Generic;
namespace LINQTakeMethodDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public int Salary { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            return new List<Employee>()
            {
                new Employee { ID = 1, Name = "Preety", Salary = 10000, Gender
"Female"},
                new Employee { ID = 2, Name = "Priyanka", Salary = 20000, Gender
"Female"},
                new Employee { ID = 3, Name = "Anurag", Salary = 35000, Gender
"Male"},
                new Employee { ID = 4, Name = "Pranaya", Salary = 45000, Gender
"Male"},
                new Employee { ID = 5, Name = "Sambit", Salary = 15000, Gender

```

```

        new Employee { ID = 6, Name = "Hina", Salary = 25000, Gender =
        new Employee { ID = 7, Name = "Santosh", Salary = 39000, Gender
"Male"}},
        new Employee { ID = 8, Name = "Sukanta", Salary = 55000, Gender
    };
    }
}
}

```

Business Requirement: LINQ Take Method with OrderBy Method

Our Business Requirement is to fetch the first four employees getting the highest salary. For this, we need to use the OrderBy method on the data source and sort the employees in descending order by their salary. Then, we need to apply the Take method and pass the count value as 4 as we need to fetch 4 employees. The following example exactly does the same thing.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQTakeMethodDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Source
            List<Employee> employees = Employee.GetAllEmployees();
            //Fetching First four Employees who are getting Highest Salary
            //Using Method Syntax
            List<Employee> ResultMS = employees.OrderByDescending(emp =>
emp.Salary).Take(4).ToList();
            //Using Query Syntax
            List<Employee> ResultQS = (from emp in employees
                                     orderby emp.Salary descending
                                     select emp)
                                     .Take(4)
                                     .ToList();
            //Accessing the Results using Foreach Loop
            foreach (Employee emp in ResultMS)
            {
                Console.WriteLine($"ID:{emp.ID}, Name:{emp.Name}, Gender:{emp.
Salary:{emp.Salary}");
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```
ID:8, Name:Sukanta, Gender:Male, Salary:55000
ID:4, Name:Pranaya, Gender:Male, Salary:45000
ID:7, Name:Santosh, Gender:Male, Salary:39000
ID:3, Name:Anurag, Gender:Male, Salary:35000
```

When to Use LINQ Take Method in C#?

The LINQ Take method is particularly useful in scenarios where you need to retrieve a specified number of elements from the beginning of a collection. Here are some common use cases:

- **Pagination:** In web or desktop applications, when you display data on pages, the LINQ Take Method can be used to fetch a specific number of items for each page. Typically, Take is used in conjunction with Skip to implement pagination functionality.
- **Data Sampling:** If you need to process or analyze a sample of data from a larger dataset, Take can be used to select the first N items from the collection.
- **Limiting Results:** When you want to limit the number of results a query returns, such as displaying only the top 10 results.
- **Performance Considerations:** In cases where processing the entire collection is resource-intensive, you might use Take to reduce the workload by processing only a subset.
- **Testing and Debugging:** During development, working with a smaller set of data can be useful to simplify testing and debugging. Take can be used to easily reduce the size of the data set you're working with.
- **Loading Initial Data:** More data will be loaded on demand when you load an initial data set on application startup or in a particular UI component.

Considerations of LINQ Take Method:

- **Deferred Execution:** Take uses deferred execution, so the operation of selecting elements is not performed until the resulting sequence is enumerated.
- **Combination with Skip:** Combine Take with Skip for effective pagination to control which sequence segment is returned.
- **Fewer Elements than Expected:** If the original sequence contains fewer elements than the specified number, Take returns all available elements without throwing an error.

Take is a simple and effective way to retrieve a specific number of elements from a sequence and is especially powerful when used in conjunction with other LINQ methods to create efficient and expressive queries.

In the next article, I will discuss the [LINQ TakeWhile Method in C#](#) with Examples. In this article, I try to explain the LINQ Take Method in C# with Examples. I hope you understand the need and use of the LINQ Take Method in C# with Examples.